in time. For example, team members should spend 10% of their time actively improving their process. Every team needs to decide how to use it, but as part of the negotiation, management can withdraw this budget for a short period of time when needed. If the budget is constantly withdrawn, it's a warning sign that another serious conversation is required.

One way an organization can encourage a learning culture is by setting up communities of practice (some organizations call these guilds) and allowing these groups to take time to discuss and share ideas during work time. Over and over, we have heard about these communities discussing books or sharing their work with other like-minded individuals.

Quality Guilds

Augusto Evangelisti, *a test engineer at Paddy Power, shares how the testers in his workplace created a quality guild.*

In my current place of work, we have a quality guild composed of 20% testers, 20% kanban facilitators, and 60% developers. Do we need somebody to tell us what to do, what problems we need to address, and what innovation we should work on? Believe me, no. We started with five people, and after a couple of months we had 20 people with incredible ideas for improving quality and making our life better. We meet every two weeks and always have a list of items to talk about.

I tried the same exercise as a manager, and I ended up with five testers who had to be there, sitting and waiting for me to tell them what to do. It is very different when the people are empowered to make their own lives better.

We're fans of Lean Coffee sessions, which encourage participants to bring up new ideas or concerns and talk openly about them. Conducting a Lean Coffee–style session that is open to everyone in an organization may lead to clearing up misunderstandings. Check the Lean Coffee links in the bibliography for Part I to get started.

Something as simple as pairing with another team member, remote or colocated, can be a learning or training opportunity, especially if you are trying to learn a new skill that someone else has already mastered.

Changing an established culture is difficult. If agile is new for your team, help team members adapt by taking small steps. Help them adopt new habits, and make sure each team member feels valued and safe.

Learn to Adapt to Change

Bernice Niel Ruhland, *a director of quality management, shares her experiences of adopting a daily standup meeting.*

Many teams call themselves "agile" but are not following the principles behind the Agile Manifesto charter. They are "agile" in spirit, but not necessarily in practices common to self-managed teams. Understand the practices and adopt what works for your team. Start with something small and let your ideas evolve naturally. Failures can lead to new information to understand where and how to apply different approaches and techniques.

For example, if you want to start a daily standup meeting, schedule 15 minutes at the same time and at the same location every day. Start with the questions typically used by agile teams: What did you do yesterday; what will you do today; and are there any obstacles in your way? Evaluate how those questions are working—perhaps for your project different questions are necessary.

Short standup meetings are a great way to keep everyone moving in the right direction and to promote face-to-face communication. Stay focused on the positive output of the daily meeting instead of defending the format. Facilitate the meetings to keep them short, so that participants know their time is valued, and modify the format if needed.

In Chapter 6, "How to Learn," we'll look at more ways teams can learn and practice the skills they need to succeed with testing in agile teams.

Transparency and Feedback Loops

Short feedback loops are an important aspect not only of testing but also of the complete agile software delivery cycle. Transparency in decision making is needed for long-term success in providing high-quality software.

If you're a line manager in a software organization, or aspire to be, learn good ways of building feedback loops and transparency into your team

culture. Read books oriented toward agile development, such as *Management 3.0* by Jurgen Appelo (Appelo, 2011). Articles and blog posts by leaders in agile organization culture, such as Johanna Rothman and Esther Derby, are excellent sources for ideas. See the bibliography for Part I for more resources.

When our work is transparent to the organization, the need for control is reduced. Visibility into what the development teams are doing helps build trust with the business stakeholders and company executives. Trust is built over time with patience and practice and consistency (see Figure 2-3).

Mastering agile values and principles requires a huge cultural shift for the organization. To quote Johanna Rothman, "Agile is for people who want to and can manage the cultural change that it requires" (Rothman, 2012a). Our real goal isn't implementing agile; it's delivering products our customers want. Spotify, a digital music service, is often cited as a company that successfully adopted an agile culture and has grown and adapted and made the whole process transparent. Henrik Kniberg has shared Spotify's experiences in an animated video that is worth watching (Kniberg and Spotify Labs, 2013).

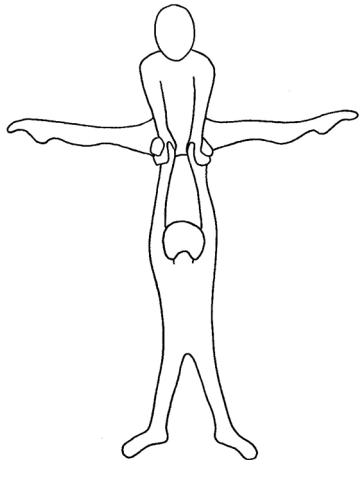


Figure 2-3 Trust takes practice and patience and consistency.

Celebrating successes—both big and small—is important. Make people aware of others' successes so that they want to be part of a culture that celebrates people's and teams' achievements.

EDUCATING THE ORGANIZATION

For a company culture to change, the top executives must be on board with changing it. This means they must understand what the change means for them.

Before you try to educate your company's executives, educate yourself about their roles and responsibilities. For example, the company CFO projects financial performance and tries to predict the return on potential investments. That person knows a lot about all aspects of the business and is likely keen to understand how software projects are progressing and what might cause delays.

Find ways to quantify and articulate the advantages of the changes you want, as well as the cost of ongoing problems such as accumulating technical debt. Show executives how the problems impact the business's bottom line.

When business leaders work with the development teams to set goals that are business driven, the teams gain an understanding of what is important. They then can make suggestions for prioritization and cutting unnecessary scope. We talk more about the idea of impact mapping to help with this in Chapter 9, "Are We Building the Right Thing?"

Stakeholders can prioritize outcomes, so the team can deliver valuable increments within the necessary time frame and use fast feedback loops to keep improving the product and giving visible feedback about the progress of projects and development.

Build trust by delivering value regularly, even if it's a small amount. As Bob Martin warns (Martin, 2011), don't say, "I'll try" when given unrealistic deadlines. Instead, help managers understand what your team can realistically deliver, quantify the cost of any shortcuts you might have to take, and help them cut scope so that they get their top-priority features.

In our experience, business executives are uncomfortable with the uncertainty that is inherent in software development. As software developers, we may be able to reliably predict delivery dates for features that are simple or that we've done before and understand well. However, stakeholders tend to want shiny new capabilities that the competition doesn't have. If the business wants something that's unpredictable, explain the complexities to them. Consider an approach such as Real Options (Matts and Maassen, 2007). Chris Matts has adapted this idea from the financial world in which options are kept open until the last responsible moment when a decision is required. Paying to extend options builds in time to learn about alternative solutions, which allows teams to make better choices. For example, we can choose technology that makes changes easier, or agree on a range of dates instead of one deadline (Keogh, 2012b).

The Cynefin framework, developed by Dave Snowden, provides a way to evaluate whether a new feature will be straightforward to deliver or has unknowns that need to be explored before a solution can emerge. See the links in the bibliography for Part I for more information about ways to "embrace uncertainty" (Keogh, 2013b).

Whether or not you use a defined approach or framework for managing risk and uncertainty, teams can help company decision makers understand why we often can't reliably predict progress, and how testing can help keep them informed.

Janet's Story

One organization I worked with found a very creative way of getting executives to understand some of the teams' issues very quickly. There were multiple teams, all working on the same product, so they had crosscutting concerns. All the daily standups were held at either 9:15 or 9:30 a.m. At 10:00 a.m., they held a Scrum-of-Scrums in a conference room where all the ScrumMasters talked about those crosscutting concerns. At 10:15, one ScrumMaster stayed behind, and any executive who had a vested interest in the product came into that room. They discussed any new issues and then reviewed outstanding issues that were assigned and dated and listed on a whiteboard. If an issue had been resolved, it was wiped off. Any new issues that had to be dealt with outside the 15 allotted minutes were added, assigned, and dated.

The book *Fearless Change* (Manns and Rising, 2005) explains good ways to introduce new ideas. For guidance while meeting with executives, check out the pattern "Whisper in the General's Ear" (pp. 248–49).

Whenever you interact with people who are not part of the delivery team, take the opportunity to educate them about how you are working. Ask them what they might need to know to do their jobs, and be willing to articulate what you might need from them.

MANAGING TESTERS

In *Agile Testing*, we talked quite a bit about test managers and what their new role could be. However, we still get a lot of questions about it and see frequent conversations about this topic on different forums. There is no one right answer—so much depends on your context. For example, if you have a small organization with only a couple of teams, you may not require a separate line manager at all.

We know of teams where all members report to a delivery or development manager, and it works very well if the manager has a good appreciation of what testers do. Augusto Evangelisti's story of guilds earlier in this chapter illustrates this approach. In his company, senior testers mentor new hires, and they all work on improving their system. Currently they have six teams, and Augusto feels strongly that as they grow, their culture will enable them to maintain this way of working.

Some organizations may need a secondary reporting structure, or someone specifically to help the learning between teams. Adam Knight shared his experience of this with us. For him, the role of a test manager extends beyond the scope of the day-to-day work of the agile team. Much of his work involves looking outside of the iteration activities and more at the cultural and strategic needs of the testing operation. He thinks it is necessary to have someone representing testing at the same level as the development director, to ensure a balanced approach that considers each discipline equally. Often in larger organizations, the disciplines are more specialized, so this may make sense. Adam also works to ensure that the organization has the appropriate balance of skills across the testing operation and that the testers in the agile teams are working collaboratively and not duplicating effort.

A test manager can also be a type of coach for the communities of practice within an organization. The test manager is not there to prescribe but can facilitate learning sessions where interested parties can discuss new ideas. As we said, there is no one right way. Understand your context and what you need, so you can practice agility and get the most benefit for your organization.

Reach out to the global software development and testing communities for help in making cultural changes that support building quality in, delivering value frequently, and working at a sustainable pace. We will mention some helpful online communities in Chapter 6, "How to Learn."

SUMMARY

This chapter was about the importance of changing and adapting organizational culture. We stressed some ideas that need nurturing at a team and organizational level:

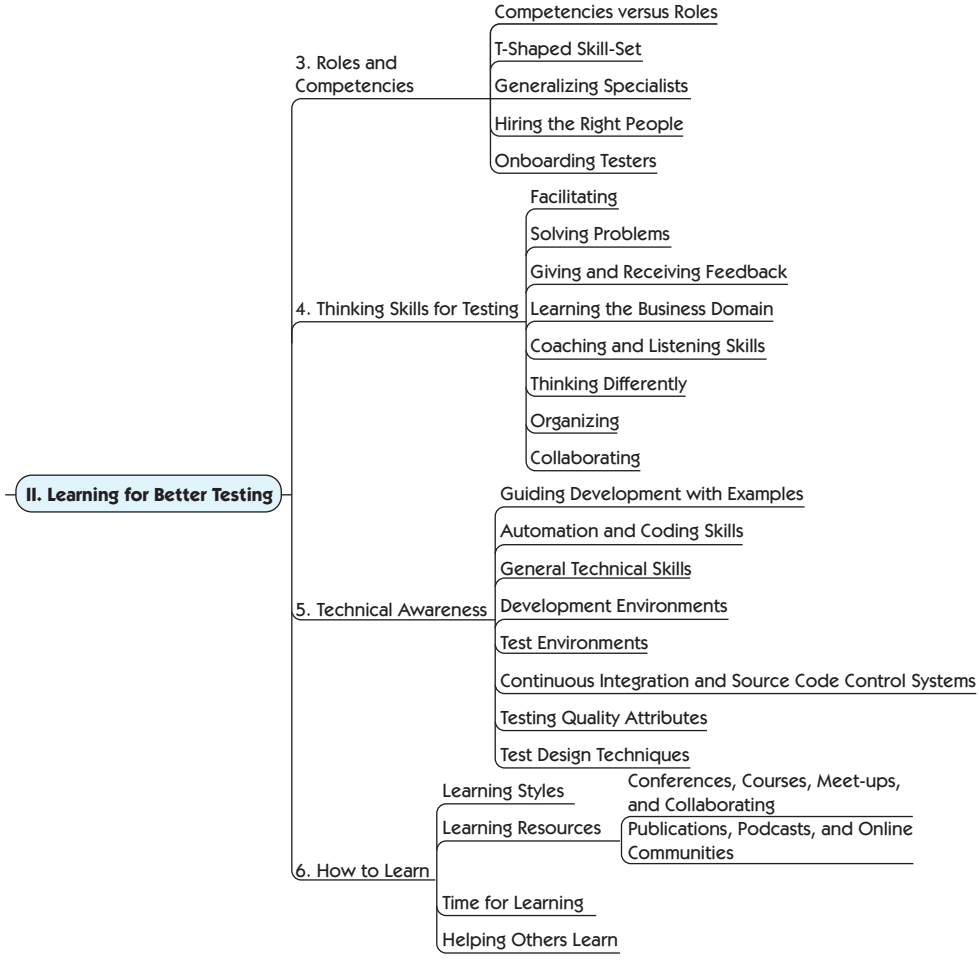
- Focus on quality, which leads to long-term sustainable pace and consistency of delivery in software development.
- Build slack into your workload by improving software delivery capability and managing scope, so your company can respond to opportunities.
- Nurture a learning culture where teams identify impediments to quality and have time to try small experiments to improve testing practices and processes.
- Educate stakeholders about how good practices pay back for the business.
- Make your process visible to help build trust between executives and delivery teams.
- Remember to celebrate successes, big and small.

Part II

Learning for Better Testing

As the years go by, we encounter more and more teams where people in all roles, not only the testers, engage in testing activities. At the same time, we've found that as testers, we need to broaden and deepen our range of skills in order to help our teams deliver the value our customers require.

In Part II, we'll explore team roles and competencies related to testing and quality, along with the types of skills needed to create high-quality software, from both a "thinking" perspective and a "technical" perspective. The last chapter in this part emphasizes the importance of learning for individuals and teams.



- **Chapter 3,** "Roles and Competencies"
- **Chapter 4,** "Thinking Skills for Testing"
- **Chapter 5,** "Technical Awareness"
- **Chapter 6,** "How to Learn"

Chapter 3

Roles and Competencies

The range of testing challenges that any team may encounter seems to widen on a daily basis. Your team may work on a web-based product and suddenly, your company decides it also needs an app that works on mobile devices. Teams adopt new technologies, development practices, tools, and frameworks while, at the same time, customer priorities shift. It follows that testing activities will have to change, too.

One of the ways teams handle these changes without constantly being in chaos is by managing what and how they learn.

Self-Managed Teams

Bernice Niel Ruhland, *a director of quality management, shares her thoughts about self-managed teams.*

We often hear about the benefits and success of self-managed teams. For some people the change can be liberating, whereas others might find it challenging. Providing a team with a project and letting them determine how to manage the work tasks can be daunting. I have encountered self-managed teams that cannot move forward without an assigned leader. Some people look to be assigned tasks; others do not feel comfortable telling other people what to do; and sometimes one or two people end up with the bulk of the work.

This does not mean these teams cannot make the transition to being self-managed; they just need help. I find it helpful to hold an initial team meeting to discuss the project, answer questions, and help the team set ground rules, with a manager, coach, or facilitator present. As a facilitator, I can work with them to define milestones but allow them to determine how they will work together. If you're in a lead or managerial role, try to delegate most of the responsibility to the team. If they are struggling with self-organizing, have a coach periodically attend a meeting to help guide when needed. However, as soon as it is reasonable, turn the meeting back over to the team.

Sessions where each team member discusses his or her skills and potential contribution to the project have also proven useful for my teams. Often people have hidden skills of which their peers are not aware. As an example, a tester may have experience with a programming language that could be used to automate some repetitive work. Progress the conversation past technical skills to include skills such as time management, conflict resolution, and relationships internal and external to the team that might be beneficial. If helpful, create a skills matrix to get the team started. Over time, shift the focus from documented skills to building relationships within the team.

Building trust between team members is even more important for a self-managed team. Some teams share common interests and create personal relationships. There are many different types of team-building activities; just be sure to use techniques that the team respects.

During retrospectives, ask what works to reinforce good behavior and discuss what team members find challenging to help make changes. Be sure to celebrate the successes and help to facilitate change when needed.

COMPETENCIES VERSUS ROLES

We've seen a positive move toward emphasizing competencies in a team rather than roles or titles. As teams make that change, we see fewer "It's not my job" excuses and more "How can I help?" conversations. Team members will continue to have core competencies in some areas more than others, but they may not identify as strongly with a particular role. For example, saying, "I am a tester" really means, "I perform mainly testing activities because that is my primary passion and strength. I can provide leadership and guidance to others, and I may also help in other areas."

Are Titles Important?

One day at a conference, **Pete Walen***, from Michigan, USA, was giving out his business card, so Janet happily accepted it. However, his title caught her attention, so she asked him about it. Here's his response.*

My business card has three different things on it, other than my name and contact information.

The obvious item is "Software Tester." That is what I do, in one form or another. I test software and work with people who also test software and help them do a better job of testing.

"Software Anthropologist" was added after much consideration. It was finally added at CAST (Conference of the Association for Software Testing) in 2011. Michael Bolton gave a talk on software development, and testing in particular, as a social science. This triggered consideration in my mind and set other things in motion. Crucial among them were questions around interaction among and between software applications and how people interact with the same software. In evaluating how software behaves, these interactions are important and form an integral part of what makes testing, testing.

This brings us to the third and most important item, "Question Asker." It seems quality assurance, QA, is a term that often gets mixed up with testing and has for as long as I have been in software development. This has irked me for some time. In 2009 I found myself in a conversation with Michael Bolton, Fiona Charles, Lynn McKee, Nancy Kelln, and a few others. In the course of the conversation, the idea of "testers asking questions that lead to information on how the software behaves or is expected to behave" kept floating in and out. Asking questions leads us to information about the software, how we interact with it, and, ultimately, more questions—at least, more questions until all the questions of interest to us and the stakeholders have been asked and answered.

Which brings us back to "Software Tester."

We like the term *question asker*, which can also be useful in planning sessions. See Part IV, "Testing Business Value," for more on this subject.

Teams in which Lisa has worked have blurred the lines between roles. Some programmers are experienced testers. Sometimes it's a tester who

comes up with a simple solution to a tricky code design problem. In addition, individuals with a wide range of competencies fill more than one role. For example, Lisa's current team has coders who also do system administration and have operations duties.

In the past few years, terms such as DevOps have gained wider use, indicating the interdependence of software development with infrastructure and operations. Although the terms may be newly popular, doing work normally done by someone in a specialized role has been a hallmark of agile teams. (We will explore the mutually beneficial collaboration between DevOps and testers in Chapter 23, "Testing and DevOps.")

Forget Developers in Test; We Need Testers in Development

Trish Khoo, *a test engineer originally from Australia, shares her experiences of what happens when the whole team thinks about testing.*

Last year I started working on a small team with developers who really value testing and treat it as an integral part of the development cycle. I'll never forget one of our first planning meetings when we were discussing a feature that we were about to build and a developer frowned and said, "Yeah, but how are we going to test it?" As a result, they changed the whole design.

I think I just about fainted with shock, having never heard that in my career. The key part of this was that it wasn't the team asking me, the tester, how *I* was going to test it. It was a question asked of the whole team: "How are we as a team going to test it? How can we build this so that we have confidence that it will work as we build it?"

As we built features, the developers would always write tests—everything from unit tests to browser-level tests—and have another developer do some manual testing before it was passed to me for testing in a fully integrated environment. By the time it was handed to me this way, I rarely found bugs that were due to carelessness. Most of the bugs I found were due to user scenarios or system scenarios that hadn't been thought of earlier.

So you might think, as a tester, did I really have much to do on a team like this? My most valuable asset in this process was still the way I thought about the product from a testing standpoint, from a user standpoint. What I found was that my testing expertise became less valuable at the end of the cycle and much more valuable at the start.

> The more effort I put into testing the product conceptually at the start of the process, the less effort I had to put into manually testing the product at the end because fewer bugs would emerge as a result.
>
> I just want to break that last bit out into a big fancy-font quote here because I think it's quite important. ***The more effort I put into testing the product conceptually at the start of the process, the less effort I had to put into manually testing the product at the end.***
>
> But the key part of this was that I knew that the development team was testing the product effectively by thinking about testing, writing tests, and manually testing as the product was being developed. I could trust that if we had thought of a scenario during planning, it would be effectively developed and tested with automated regression tests in place by the end of the process.
>
> I've had a lot of discussions this year around the role of the tester. Let's put that aside for now and start thinking about the role of a software developer. A software developer needs to be able to build a product with confidence that it does what it's expected to do. Knowing how to do that at a basic level should be critical to the role of a good software developer. For that reason, we need more testing in software development. And it needs to be done by the people who are building the product.
>
> Having a testing specialist on the team is a valuable asset, but the responsibility for testing shouldn't be restricted to a single person. You might also have a database specialist on your team, but it doesn't mean that he or she is the only person working with databases. The same goes for testing. The specialist can help with the really difficult testing problems, knowing that the rest of the team is capable of dealing with the simple testing problems.
>
> Then it's shorter feedback loops, greater confidence, and faster quality releases for all. Who doesn't love that?

We explored the interaction between customer and developer teams in *Agile Testing*. Since then, many agile teams have incorporated specialists with different competencies. For example, it's more common now to find business analysts on an agile team, as well as testers who are doing a lot of business analysis activities. Boundaries between roles continue to blur. At the same time, creating a cross-functional team doesn't mean dispensing with specialties. It's been our experience that teams often feel blocked because a particular skill set is missing from their team.

Sometimes the solution is simple: hire someone with those skills, or find ways to train existing team members.

Lisa's Story

At my last job, my team became frustrated because we'd start a new iteration with several stories that were not well defined by the business experts at the parent company. As a result, we were constantly going to the product owner for clarification of requirements, or to show him what we had developed, only to be told it was wrong.

We testers suggested hiring a business analyst (BA) who could work with the product owner to help the customers better articulate what they needed. Our product owner was unfamiliar with the new parent company, and although he met with the stakeholders, he didn't know the right questions to ask to get to the heart of the features they wanted. A skilled BA would know how to collaborate with the customers and ask the right questions.

Unfortunately, the manager wasn't willing to hire a BA, so we formed a business analysis community of practice. Testers, the product owner, the ScrumMaster, the development manager, and interested programmers got together and identified ways to build our analysis skills. We read books and articles, attended conference workshops, and participated in webinars to gain competency in business analysis. We met at regular intervals to share information and documented it on the team wiki.

It might have been better to hire an expert, but our efforts paid off. The product owner learned techniques to use when meeting with stakeholders from the parent company, so he understood the desired features better. We still sometimes lacked context for what business problems the customers were trying to solve, but we no longer spent so much time going back and forth to the product owner for information on the stories we were developing.

T-SHAPED SKILL SET

In *Agile Testing*, we described ten principles for agile testers, which emphasized attitude and mindset over specific technical skills. As a quick review, they are:

- Provide continuous feedback.
- Deliver value to the customer.
- Enable face-to-face communication.
- Have courage.
- Keep it simple.
- Practice continuous improvement.

- Respond to change.
- Self-organize.
- Focus on people.
- Enjoy.

However, we still hear the question, "Do testers on agile teams need to be programmers?"

Our answer is that testers need T-shaped skills (see Figure 3-1), a term first defined by David Guest (Guest, 1991). To work effectively on any given team, we need both broad and deep skill sets. Broad knowledge in areas other than our own specialty enables us to collaborate across disciplines with experts in other roles. Deep knowledge and extensive practice in a single field ensure that we bring something essential to the team. Rob Lambert has a nice blog post on this subject (Lambert, 2012).

The top of testers' "T" typically includes technical skills such as a basic understanding of their system's architecture, knowledge of general programming concepts and design principles, ability to do basic database queries, and competence with such tools as integrated development environments (IDEs) and continuous integration (CI) dashboards.

Team members in other roles need basic testing concepts among other skills in their T-bar. Shallow domain knowledge may be adequate in

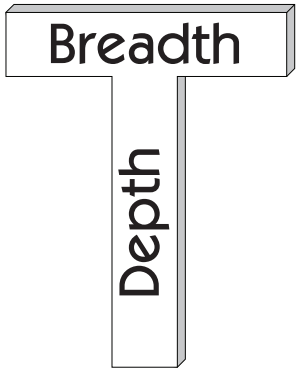


Figure 3-1 T-shaped skill set

some situations. In others, the ability to deliver business value requires that some team members, perhaps testers or business analysts, need to know the business in depth. Get the whole team together to talk about T-shaped skills and how to fill in any gaps. And don't forget those ten principles for agile testers. Attitude really is everything.

Square-Shaped Team

> **Adam Knight**, *a director of QA and support in the UK, shares his story about his experience with T-shaped team members to make a square-shaped team.*

My team and I have been working on testing a Big Data storage system for seven years. The system is a large-scale data storage system combined with a SQL query engine that runs on various operating systems, primarily Linux. One of the major problems that I have encountered in testing a product of this nature is the range of relevant skills that are required to perform all of the tasks necessary to test the system. The testers in our company needed the ability not only to test the product from the perspective of the various stakeholders, but also to build and maintain the various testing environments and architectures that were needed. Some of the skills that we required were

- Operating system knowledge in Linux/UNIX to create and maintain test environments and monitor those environments to assess the impact of the software upon them
- Virtualization and cloud knowledge to expand the testing into environments that could support the multiple machine clustered environments
- Scripting knowledge to continuously develop and maintain the numerous harnesses required to test the product via the server command-line tools
- Programming knowledge to develop and maintain the harnesses required for functional and scalability testing of the client API interfaces where these differed from our core product programming languages of C and C++
- SQL and database knowledge to understand the implementation domain and test the extensive SQL engine with a range of realistic queries
- Exploratory testing skills to identify and exercise the range of states and combination of operations that can affect the data storage layer

It became apparent very quickly that we were unlikely to find such a wealth of skills in one individual. Instead, I took the approach of trying to populate the team with a number of testers who, in combination, possessed the range of skills that would allow the team to address the multiple challenges that it was facing. Each individual would need to have a general set of abilities to understand the product and the general testing approach.

As we brought on new team members, we made sure that each one possessed a deep set of abilities, which included the priority skills the team had identified. These abilities were complementary to those of the other team members and to those of the team as a whole.

When I first read about the concept of the "T-shaped" individual, it resonated with what we were doing. The idea of a broad set of general skills combined with a deep core of specialist abilities was exactly the shape of individual that we had tried to recruit. So, for example, I was already very fortunate to work with one tester who possessed solid database and SQL language knowledge combined with domain knowledge from a previous role as a database administrator (DBA).

We employed one individual with strong scripting skills to maintain the core harness, backed up with excellent exploratory testing abilities. Another had great operating system abilities to create test environments, virtual clusters, and Hadoop and cloud testing, combined with knowledge of soak and performance testing. Figure 3-2 shows how each tester has different depth of skills with the same breadth.

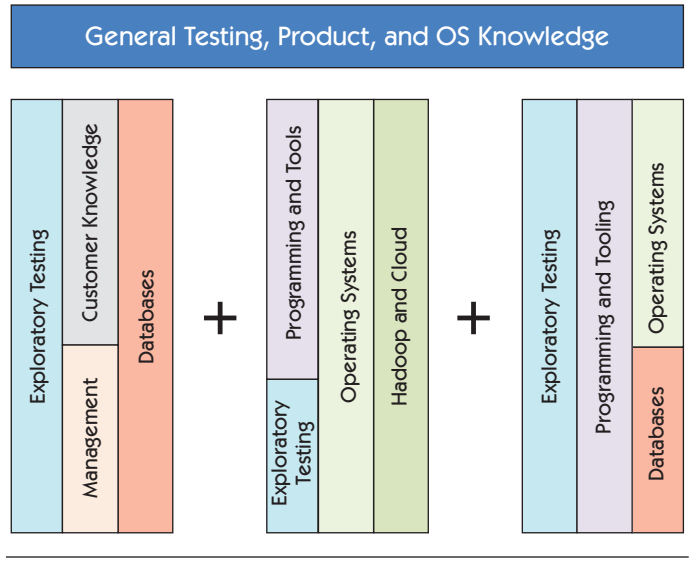


Figure 3-2 Three testers with different strengths

I felt that the T-shaped concept was missing one of the core reasons for our taking the approach that we did. The concept of the T-shaped individual is limited to exactly that: an individual. What I felt was the true power of the T-shaped tester was when those individuals combined their abilities into a team in the manner that we had done, a concept I labeled the "square-shaped team," as shown in Figure 3-3.

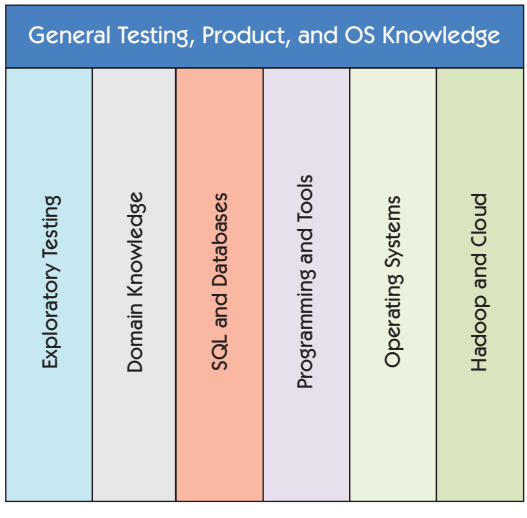


Figure 3-3 Square-shaped team

Each tester we've hired has brought unique skills to the team. Some of these skills we knew were a priority when we were in the process of hiring; others were less apparent at the time but still valid. For example, one tester we hired was less of an obvious choice, having a consulting background working on implementation projects. This meant, however, that his abilities in reporting and requirements analysis made him an excellent addition for understanding stakeholder concerns and defining acceptance criteria accordingly. I've also used his exceptional test reporting as an exemplar to the rest of the team. If we had looked at such an individual in isolation, he or she may not have been considered for a testing role on the product. By taking a holistic approach to building the team, we could integrate new members well, and all would benefit from the unique perspective that each one had.

Like Adam, we think it is important to look at the team skills as well as the individual. Together the whole team can solve most anything. Don't be afraid to look around you and use the knowledge that other people bring to the table.

GENERALIZING SPECIALISTS

Agile development attracts generalizing specialists. This term has been used to mean individuals with a deep level of knowledge in at least one domain and an understanding of at least one other. Hmm, sounds a lot like T-shaped skills. Sometimes the term is used to describe a person who is good at everything, but that is not what we mean. We run the risk of diluting our strengths, and instead of doing a few things well, we do many things with mediocrity. However, in order to collaborate well with team members in other roles, we need to know some basics of their specialty.

Becoming a Generalizing Specialist

Matt Barcomb, *an organizational design specialist, shares his ideas about how a generalizing specialist evolves.*

It's one thing to know what a generalizing specialist is, but how do you actually go about becoming one? Most people spend a lot of time becoming very good at their specialty but don't understand how to generalize effectively without simply becoming a specialist in another area.

What a Generalizing Specialist Isn't

I've been to a number of places in the past few years where management glommed onto the term too excitedly. The enticing but misguided belief was that programmers, testers, designers, analysts, technical writers, release engineers, and just about everyone else (except the managers) would all eventually know how to do each other's work and could be swapped, interchangeably, for any other person on the team. Imagine how simple the resource-balancing spreadsheets and project plans would become!

That situation is not a team of generalizing specialists; it would be a team of generalizing generalists. It is also pure fiction, as it would be impossible for all the people involved in releasing enterprise software to know every other person's function with the depth that would be needed to deliver effectively.

So What Is a Generalizing Specialist?

Being a generalist on a cross-functional team means being able to effectively appreciate, communicate with, and collaborate with other team members and roles with different specialties. It does not mean the person can do the same work as different specialists with equal skill or enthusiasm.

For example, a programmer with a deep knowledge of code craft is not interchangeable with a tester and vice versa. However, both might be able to collaborate, as a pair, on either person's tasks. Ideally the difference is obvious and the example could just as easily be applied to any other role on the team, such as tester, designer, analyst, operations, technical writer, or even manager!

How to Become a Generalizing Specialist

There are many approaches to becoming a generalizing specialist. There is no one right way to learn, and ultimately becoming a generalizing specialist is all about learning!

I approach becoming T-shaped as a skill acquisition problem. There are many models for skill acquisition, but for the purpose of becoming a generalizing specialist I use three categories: (1) Basic, (2) Advanced, and (3) Meta, combined with an associated amount of "stuff to learn." The amount of stuff to learn varies based on the category; Basic has a small amount, Advanced has the most significant amount, and finally Meta has a lower amount, but more than Basic.

This might look like the model shown in Figure 3-4.

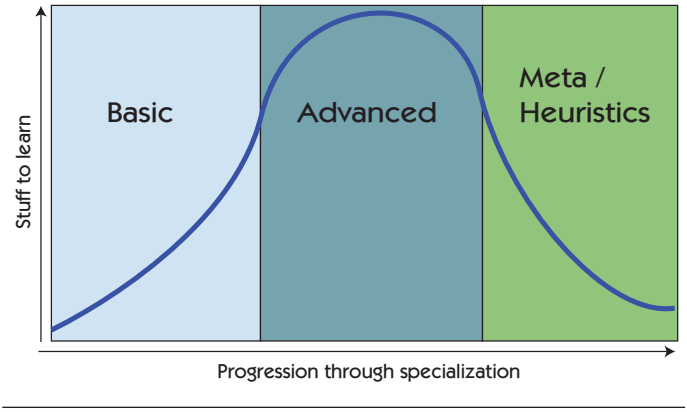


Figure 3-4 Model for becoming a generalizing specialist

This model actually shows how to become a specialist. However, it also offers hints for how to become a generalist. According to the model, every beginner has to learn the basics of a skill or specialty (like testing or programming). This is equally true for generalists or aspiring specialists.

So, the first step to becoming a generalist is that you have to learn the basics of the specialty. For a tester on a cross-functional team, this might mean the technical awareness that Lisa and Janet refer to in Chapter 5, "Technical Awareness." For example, testers might learn more about the deployment pipeline or target environments; they might learn about how to query a database, or about the syntax, structures, and tools the programmers use for developing the product.

The specialist then spends years (sometime the majority of a career) learning the advanced subject matter of his or her specialty and even longer to derive the Meta knowledge about the specialty. Obviously, having everyone on a cross-functional team spend so much time to learn a specialty is not the most effective way to become a generalizing specialist. So the question is, How can an individual continue to grow as a generalist but skip over the years required to progress through Advanced and into Meta?

The answer to the question lies in understanding the nature of the derived Meta skills. Meta knowledge of a specialty is that intuitive, tacit knowledge one develops after years of applying advanced knowledge from a specialty in many different situations—almost a sixth sense within a domain. A specialist might describe something as "just not feeling right" or the "shape" or "smell" of something like the code, design, or architecture of a product. The point is that there is something the specialist is sensing, some sign to look for.

Such signs can be taught to non-specialists as heuristics or rules of thumb and is the next way a generalist can grow. A generalist can pair with a specialist and offer valuable help with signal detection of potential problems. The heuristic may not always be right, but that's OK; that's how rules of thumb work. So, while knowing how to correctly apply Meta knowledge in a given situation can take years to develop, generalists can be taught the heuristic signals to look for.

Some examples of this for a tester generalizing toward programming might be the rules of clean code (Martin, 2009), the principle of DRY (don't repeat yourself), length or counts of classes, variables, function arguments, or too many if-statements in a method. The generalist would not need to know how to fix these things, and sometimes there may be good reasons that a rule is being broken. The value-add is helping the specialist with signal detection, not problem correction.

The reason this is valuable is because individuals tend to engage at a lower level or in a more focused and analytical way when executing a task or implementing something. Applying Meta knowledge or looking for heuristics requires a higher-level view or a more abstract way of thinking about the task at hand. It is difficult for a single person to perform a task in both an analytical and an abstract fashion simultaneously.

So, when generalists learn the basics and, more importantly, the heuristics of a specialty, they are able to more effectively communicate with those specialists. This is a very important first step in having an effective cross-functional team. Moving from communication toward true collaboration is key for a more mature cross-functional team.

The concept of generalizing specialists applies to all roles on the team, not only testers. When programmers learn testing basics, they can communicate more effectively with testers and learn better ways to prevent defects and build quality in as they develop the software. In our experience, pairing with a tester is the easiest way for programmers to build up their testing knowledge.

Lisa's teams benefited from documenting checklists and specialized testing information on the team wiki. While information on a wiki is no substitute for face-to-face conversation, it can serve as a memory jogger.

In Chapter 6, "How to Learn," we will explore ways that testers can learn the basics of other specialties such as programming and database design so that they can communicate more easily with teammates in other roles.

HIRING THE RIGHT PEOPLE

Lisa was influenced early in her agile career by Alistair Cockburn's paper "Characterizing People as Non-Linear, First-Order Components in Software Development" (Cockburn, 1999). After studying dozens of software projects over 20 years, he found that the common success factor in software development was that "a few good people stepped in at key moments." Having good people made projects successful, not the programming language, tools, or methodology used.

In our experience, it's critical to get people with the right attitude for the team. Take the time you need to find testers who are a good fit. If you hire people who express curiosity, want to learn, and aren't shy about going out of their comfort zone, they can be trained for your specific needs. Although colocation has great advantages over distributed teams, consider enlarging your geographical search. Testers working remotely can be effective if the team is disciplined about using good communication practices and takes advantage of today's technology.

Hiring Geeks That Fit by Johanna Rothman (Rothman, 2012b) offers more help on finding and hiring testers with cultural fit and the abilities your team needs. Each individual brings his or her strengths and unique perspective, and it's important to look at all aspects of the value each individual contributes. Diversity within a team is essential to get different perspectives. Sometimes the person who isn't a good fit at the beginning, but is passionate about delivering a quality product for the customer, may still play a valuable role.

ONBOARDING TESTERS

Even experienced agile testers need help finding their place on a new team. We find it helpful to set expectations. What can that person expect to know by the end of the first day? The first week? The first month? Put this information on a wiki page or some other easily accessible and maintainable location.

Help the new hire get to know members of both the development and customer teams. Provide a list of "who knows what," and introduce the new person to each of those people. Schedule time for the new person to sit with business experts and find out what they do in their jobs. Take steps to ensure that the new tester has plenty of time to learn and doesn't feel pressure to "produce" right away. Help the new hire feel safe about asking for help and posing questions at any time.

In our experience, pairing is the best way to bring new people up to speed. Pair the new tester up with other testers, with programmers, with BAs, with DevOps, with business experts. It's helpful to assign a buddy or mentor as the first line of support for the new hire; the two can act

as a default pair. There's a bonus to pairing with the new person who provides a fresh set of eyes.

Lisa's Story

I was pairing with a highly experienced tester who was new to our financial services domain. One of our test results came out a few pennies different from the expected result. I'd seen this many times over the years and wrote it off to a rounding difference. But the discrepancy bugged our new tester. He went over to talk to one of the programmers about it.

It turned out to be an actual bug that we had all been ignoring for years. Tests were written, the problem was fixed, and we no longer saw the discrepancies. Pennies add up, so I feel bad thinking that some of our customers may have been slightly shortchanged! Nothing beats a fresh set of eyes. Lesson learned: don't discount anything a new tester observes!

Be creative as you plan training for your team's new tester. Set up a session with someone who can explain the system architecture. Schedule time for a programmer or system administrator to walk the new tester through the CI process. Provide training on the automated test suites and the living documentation that these provide. One-to-one discussions add tremendous value to the regular practice of reading all the documentation because the new employee has an opportunity to ask questions. New team members may get overwhelmed with the complexity of trying to learn everything at once. Try breaking the training into smaller chunks. Perhaps after a session of training, give them a charter to explore that area. There's more on charters and exploratory testing in Chapter 12, "Exploratory Testing."

Mike Talks told us that he tries to spend 15 to 30 minutes every day for the first two weeks catching up with new employees. After that, he drops it to weekly, and then monthly. It will be many months before they're up to speed, so don't try to rush the process. Everyone benefits from a thoughtful approach to bringing along a new tester.

Summary

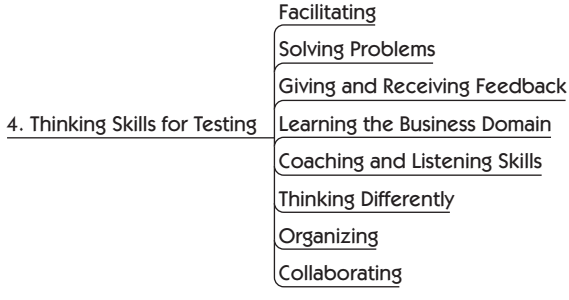
Teams that succeed in creating high-quality software include people in a variety of roles and with a wide range of competencies. Look for ways to get the ones you need on your team.

- Teams whose members have a variety of T-shaped skill sets succeed with testing in fast-changing environments.
- All team members need broad agile testing basics, allowing them to collaborate well for improving quality, but each one may contribute a different deep, specialized skill.
- Testers communicate better with programmers, business analysts, product owners, managers, DevOps practitioners, and other team members when they know the basic concepts of those other specialties.
- Hire team members who are passionate about quality and learning, whose T-shaped skills complement each other's.
- Set realistic expectations for new hires, and build in visibility and feedback.

This page intentionally left blank

Chapter 4

Thinking Skills for Testing



The *Oxford English Dictionary* defines soft skills this way: "personal attributes that enable someone to interact effectively and harmoniously with other people." "Soft" seems to imply that these skills are easier to learn or less important than "hard" skills such as technical competencies. There's a lot of debate over the best term to describe these abilities. Some like to call them people skills. We prefer the term *thinking skills*, because in addition to our relationships with people, these skills apply to other areas such as problem solving, understanding the business domain, using the right thinking style for a given testing activity, and organizing our time.

Thinking skills are not tangible in the sense that we can say, "I've learned that; I can practice it perfectly now." Abilities such as communication, collaboration, facilitation, problem solving, and prioritization can be the most difficult to master, yet they are the most crucial for success in agile testing.

In many organizations, when testers are part of a separate team, they tend to talk only to other testers, possibly to programmers, but rarely to anyone on the customer team. On agile teams, however, testers and other team members work closely with business stakeholders and product owners and managers to elicit requirements and uncover hidden

assumptions. Programmers, analysts, and team members in other roles also contribute to eliciting requirements, helping to address the impact of technical issues and dependencies. This is why concepts such as systems thinking—how we got here, and what changes impact other parts of the system—are so critical.

Testers and other team members engaged in testing and quality-focused activities can apply interpersonal and leadership skills to help the development and customer teams improve their software product and process.

FACILITATING

Activities such as specification workshops (Adzic, 2009) work best when someone skilled in facilitating helps to guide the discussion. A facilitator (see Figure 4-1) who isn't engaged in capturing the requirements would be ideal, but any team member who has key thinking skills, such as how to get people in different roles to work together and stay focused on a common objective, can step up if needed. Specification workshop facilitators help stakeholders set business goals and help both the development and customer team members collaboratively define the scope that will achieve those goals.

Similar skills help you facilitate informal brainstorming sessions, ensuring that each participant feels free to express ideas without being

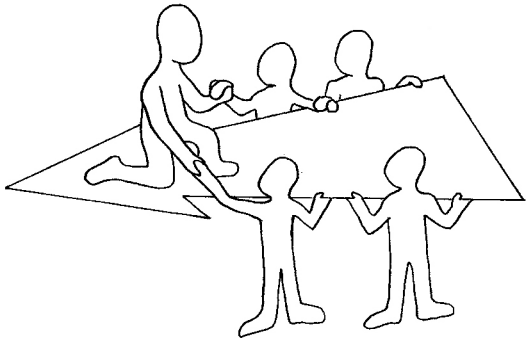


Figure 4-1 Facilitation helps teams gain common understanding.

criticized. Working on acquiring these skills helps you find creative ways for your team to solve problems. See the bibliography for Part II, "Learning for Better Testing," for more recommended books to help you hone your facilitation skills for gathering requirements and collaborating effectively.

SOLVING PROBLEMS

When we talk about nontechnical skills, we don't mean they are easy to master. For example, many of us continually work to improve our problem-solving skills; some university computer science or IT programs may teach skills like this, but often they aren't in the curriculum. People generally need to acquire them on the job.

Janet's Story

I remember when I first wrote my exam for the ASQ Certified Quality Manager, I failed the written part, which was how to address two specific problems. I believe I failed because I didn't remember how to apply my problem-solving skills. I first learned those skills when I took Physics 101 at the University of Alberta, but I didn't apply them on a regular basis. When I failed my exam, I sat back, figured out the root cause, and went back to basics by reviewing how to solve a problem. In physics, that started with drawing a picture. I rewrote my exam following the principles of problem solving and passed. I then presented what worked for me to the next group of people who wanted to write the exam, which reinforced what I had learned.

Problem solving is one of those transferable skills that can be applied to test design, debugging, coaching, or teaching. Perhaps the most useful thinking skill is to know how to help your team address its problems, rather than going in and fixing the symptoms. Courses such as Problem Solving Leadership (PSL) (Derby et al., 2014) are a good way to learn how your team can reframe problems, resolve conflicts, and communicate more effectively. You don't have to be in a management position to provide leadership for your team and help them improve their problem-solving effectiveness.

Tools that help us visualize our thinking, such as mind mapping, impact mapping (see Chapter 9, "Are We Building the Right Thing?"), and